

Decoding the Mind of Neural Chess Engines

A Unified Theory of Transformer Attentions, Monte Carlo Tree Search,
Interpretability Probes, and AI-Human Coaching Systems

Chess Speak Out Loud Project Team

Sourced, Synthesized & Formatted for Theoretical Completeness

August 6, 2026

Abstract

This treatise presents a comprehensive, self-contained mathematical and architectural theory governing modern neural-network-guided chess engines, specifically focused on Leela Chess Zero (LC0) and its integration into interactive human coaching systems. We formally derive the Monte Carlo Tree Search (MCTS) framework and its governing Predictor Upper Confidence bounds applied to Trees (PUCT) formula, tracing its roots from classic multi-armed bandit algorithms (UCB1) to deep reinforcement learning. We explore mechanistic interpretability inside transformer representations, detailing how linear probes extract implicit future look-ahead activations (3–7 plies ahead) and detect "suppressed win" sacrificial continuations. Furthermore, we analyze alternative engine steering paradigms—including Reward Shaping, Multi-Objective RL, Intrinsic Motivation (Entropy Maximization), Dense Mobility Rewards, and Guided Trajectory Rollouts. Finally, we address the human-AI translation layer, introducing the Policy-MCTS Divergence Index ($\Delta_{\text{Policy, MCTS}}$), relational fact extraction, and the Saliency Problem for grandmaster-level automated commentary.

Contents

1	Foundations of Neural Chess Architecture	3
1.1	The Paradigm Shift: From Minimax to Neural Intuition	3
1.2	The LC0 Transformer Architecture (BT3-768x15x24h)	3
1.2.1	Input Board Representation	4
1.2.2	Stacked Transformer Encoder Layers	4
1.3	Dual Output Heads: Policy and Value	4
1.3.1	The Policy Head: Move Intuition	4
1.3.2	The Value Head: Calibrated Uncertainty	5
2	Monte Carlo Tree Search & The PUCT Formula	6
2.1	From Multi-Armed Bandits to Game Trees	6
2.1.1	The Classic Multi-Armed Bandit & UCB1	6
2.1.2	UCT: UCB Applied to Trees (2006)	6
2.2	Rigorous Derivation of the PUCT Formula	6
2.2.1	The PUCT Selection Rule	7
2.2.2	Detailed Component Definitions	7
2.3	Asymptotic Analysis & The Refutation Mechanism	7
2.4	Intuitive Analogies of the PUCT Formula	8
2.4.1	The Casino Slot Machine Model	8
2.4.2	The Talent Hiring / Job Interview Model	8
2.5	The 4-Phase Search-Inference Loop	9
3	Mechanistic Interpretability & Linear Probing	10
3.1	Peering Inside the Black Box: Intermediate Transformer Layers	10
3.2	Mathematical Formulation of Linear Probes	10
3.2.1	Finding 1: Implicit Future Look-Ahead (3-7 Plies Ahead)	10
3.2.2	Finding 2: Time-Travelling Attention Heads	10
3.3	The Suppressed-Win Probe ("The Tal Detector")	11
4	Advanced Engine Steering & Reinforcement Learning	12
4.1	Hacking MCTS Search vs. Representation Steering	12
4.1.1	Search Hacking: Heuristic PUCT Bias	12
4.1.2	Multi-Objective Reinforcement Learning	12
4.2	Creating Tactical Chaos: Intrinsic Motivation & Entropy	12
4.3	Creating Positional Paralyzation: Dense Mobility Rewards	13
4.4	Latent World Models & Guided Trajectory Rollouts	13
5	Human-AI Translation & The Saliency Problem	14
5.1	The Failure of the Single Evaluation Bar	14
5.2	The Policy-MCTS Divergence Index ($\Delta_{\text{Policy, MCTS}}$)	14
5.3	Relational Board Facts & The Saliency Problem	15

5.3.1	The Saliency Problem Defined	15
5.3.2	Contrastive Objective Extraction	15
5.3.3	Learning Saliency from Grandmaster Annotations	15
6	System Architecture of the AI Coach	16
6.1	System Dataflow Pipeline	16
6.2	The Translator's Contract (Zero Hallucination Guarantee)	16
6.3	Conclusion	17

Chapter 1

Foundations of Neural Chess Architecture

1.1 The Paradigm Shift: From Minimax to Neural Intuition

For over half a century, computer chess was dominated by brute-force search algorithms. Systems such as Deep Blue and traditional Stockfish relied on hand-crafted evaluation functions paired with α - β pruned minimax search trees. An evaluation function mapped a board state s to a scalar score $f(s) \in \mathbb{R}$ by summing weighted features such as material balance, pawn structure, and piece mobility:

$$f(s) = \sum_i w_i \cdot \phi_i(s) \quad (1.1)$$

While computationally formidable, this approach suffered from two fundamental limitations:

1. **Evaluation Blindness:** The evaluation function $f(s)$ was rigid and static. It could not perceive holistic geometric patterns, complex color-complex weaknesses, or long-term dynamic compensation for sacrificed material without explicit human programming.
2. **Combinatorial Explosion:** Without strong intuition guiding which moves to examine, α - β search was forced to calculate millions of subtrees per second, evaluating vast swaths of irrelevant continuations.

The advent of AlphaZero and Leela Chess Zero (LC0) introduced a paradigm shift by replacing hand-crafted heuristics and deep uniform calculation with **Deep Neural Networks (DNNs)** trained via self-play reinforcement learning. The engine split chess playing into two unified components:

- **Perceptual Intuition:** Provided by a deep neural network that evaluates positions and predicts move probabilities in a single forward pass without searching.
- **Calculated Verification:** Provided by Monte Carlo Tree Search (MCTS), which uses the network's intuition to prune the search tree and verify lines deeply.

1.2 The LC0 Transformer Architecture (BT3-768x15x24h)

Modern flagship LC0 networks, such as the BT3-768x15x24h architecture used in our diagnostic pipeline, depart from classic Convolutional Residual Networks (ResNets) in favor of **Vision Transformers (ViTs)** adapted for spatial board topologies.

1.2.1 Input Board Representation

A chess position s is converted into an input tensor $\mathbf{X}^{(0)} \in \mathbb{R}^{64 \times d}$ comprising 64 spatial tokens (one for each square from $a1$ to $h8$), where each square is embedded into a $d = 768$ -dimensional vector space. The input features for each square include:

- Binary piece placement planes (Pawn, Knight, Bishop, Rook, Queen, King for both White and Black).
- Repetition counters, castling rights, and side-to-move indicators.
- Historical position planes (to capture en-passant and pawn movement momentum).

1.2.2 Stacked Transformer Encoder Layers

The input tensor $\mathbf{X}^{(0)}$ passes sequentially through $L = 15$ Transformer Encoder layers:

$$\mathbf{X}^{(0)} \xrightarrow{\text{Layer 1}} \mathbf{X}^{(1)} \xrightarrow{\text{Layer 2}} \dots \xrightarrow{\text{Layer } l} \dots \xrightarrow{\text{Layer 15}} \mathbf{X}^{(15)} \quad (1.2)$$

Each Transformer layer l consists of Multi-Head Self-Attention (MHSA) followed by a Feed-Forward Network (FFN) with Layer Normalization and residual connections:

$$\mathbf{Z}^{(l)} = \text{LayerNorm} \left(\mathbf{X}^{(l-1)} + \text{MHSA} \left(\mathbf{X}^{(l-1)} \right) \right) \quad (1.3)$$

$$\mathbf{X}^{(l)} = \text{LayerNorm} \left(\mathbf{Z}^{(l)} + \text{FFN} \left(\mathbf{Z}^{(l)} \right) \right) \quad (1.4)$$

In the MHSA module, each of the $H = 24$ attention heads computes an attention matrix $\mathbf{A}_h^{(l)} \in \mathbb{R}^{64 \times 64}$ over the 64 squares:

$$\mathbf{A}_h^{(l)} = \text{softmax} \left(\frac{\mathbf{Q}_h^{(l)} (\mathbf{K}_h^{(l)})^T}{\sqrt{d_k}} \right) \quad (1.5)$$

Where $\mathbf{Q}_h^{(l)} = \mathbf{X}^{(l-1)} \mathbf{W}_{Q,h}$ and $\mathbf{K}_h^{(l)} = \mathbf{X}^{(l-1)} \mathbf{W}_{K,h}$ are the Query and Key projections for head h . This allows the network to model spatial piece-to-piece dependencies across arbitrary board distances.

1.3 Dual Output Heads: Policy and Value

At layer 15, the hidden activation tensor $\mathbf{X}^{(15)}$ branches into two specialized output heads:

1.3.1 The Policy Head: Move Intuition

The Policy Head predicts a probability distribution over all $M \approx 1858$ legal moves in chess:

$$P(a|s) = \text{softmax} \left(\mathbf{W}_{\text{policy}} \mathbf{X}^{(15)} + \mathbf{b}_{\text{policy}} \right)_a, \quad \sum_{a \in \mathcal{A}(s)} P(a|s) = 1 \quad (1.6)$$

Where $\mathcal{A}(s)$ represents the set of legal move candidates in position s . $P(a|s)$ represents LCO’s 0-node ”instinct”—the move it prefers before calculating a single variation.

1.3.2 The Value Head: Calibrated Uncertainty

The Value Head outputs a Win/Draw/Loss probability vector $\mathbf{v}(s) = (w(s), d(s), l(s))^T \in [0, 1]^3$, satisfying $w(s) + d(s) + l(s) = 1$:

$$\mathbf{v}(s) = \text{softmax} \left(\mathbf{W}_{\text{value}} \text{ReLU} \left(\mathbf{W}_{v1} \mathbf{X}^{(15)} + \mathbf{b}_{v1} \right) + \mathbf{b}_{\text{value}} \right) \quad (1.7)$$

The expected scalar value $V(s) \in [-1, +1]$ from the side-to-move perspective is:

$$V(s) = w(s) - l(s) \quad (1.8)$$

Unlike scalar evaluations in traditional engines, the WDL distribution exposes the **character of the fight**: a high draw probability indicates a stable position, whereas high win and loss probabilities ($w = 45\%$, $d = 10\%$, $l = 45\%$) signal razor-sharp, high-entropy tactical chaos.

Chapter 2

Monte Carlo Tree Search & The PUCT Formula

2.1 From Multi-Armed Bandits to Game Trees

The core decision problem in MCTS is allocating a finite computational budget (search visits) among competing candidate moves. This is mathematically modeled as a **Multi-Armed Bandit Problem**.

2.1.1 The Classic Multi-Armed Bandit & UCB1

Suppose a gambler faces K slot machines ("arms"), each yielding rewards from an unknown distribution. To maximize total payout over T pulls, the gambler must balance:

- **Exploitation:** Pulling the arm that has yielded the highest average reward so far.
- **Exploration:** Pulling arms that have been tested infrequently to uncover hidden high-yield options.

In 2002, Auer et al. proved that the **Upper Confidence Bound (UCB1)** algorithm achieves logarithmically optimal regret by selecting the arm i that maximizes:

$$i^* = \arg \max_i \left[\bar{X}_i + c \sqrt{\frac{\ln N_{\text{total}}}{N_i}} \right] \quad (2.1)$$

Where $\bar{X}_i$ is the empirical mean payout of arm i , N_i is the number of times arm i has been pulled, and N_{total} is the total pulls across all arms.

2.1.2 UCT: UCB Applied to Trees (2006)

Kocsis and Szepesvári (2006) extended UCB1 to game trees in the **Upper Confidence bounds applied to Trees (UCT)** algorithm. Every node in the search tree was treated as an independent bandit problem. However, pure UCT suffered from a major drawback in chess: **it possessed zero prior knowledge**. At every new position, UCT was forced to sample every legal move—including absurd blunders like giving away a Queen—equally before focusing on promising lines.

2.2 Rigorous Derivation of the PUCT Formula

To solve UCT's zero-prior limitation, Rosin (2011) and Silver et al. (AlphaGo 2016, AlphaZero 2017) introduced **Predictor / Prior UCT (PUCT)**. By incorporating the neural network's

policy prior $P(a|s)$, PUCT biases the exploration term toward moves that the network’s intuition considers strong.

2.2.1 The PUCT Selection Rule

During the selection phase of MCTS, the engine traverses the search tree from the root node s_0 . At each node s , it selects the action a^* that maximizes the PUCT objective:

$$a^* = \arg \max_{a \in \mathcal{A}(s)} [Q(s, a) + U(s, a)] \quad (2.2)$$

Where $Q(s, a)$ is the exploitation term (mean action value) and $U(s, a)$ is the exploration term (upper confidence bound).

2.2.2 Detailed Component Definitions

1. **Exploitation Term $Q(s, a)$:** The average evaluation of all search visits that have passed through edge (s, a) :

$$Q(s, a) = \frac{W(s, a)}{N(s, a)} \quad (2.3)$$

Where $W(s, a)$ is the accumulated Value Head score ($V(s')$) backpropagated through edge (s, a) , and $N(s, a)$ is the total visit count of edge (s, a) . If $N(s, a) = 0$, $Q(s, a)$ is initialized to a default value (e.g., parent value $V(s)$ or first-play urgency FPU).

2. **Exploration Term $U(s, a)$:**

$$U(s, a) = c_{\text{puct}} \cdot P(a|s) \cdot \frac{\sqrt{\sum_{b \in \mathcal{A}(s)} N(s, b)}}{1 + N(s, a)} \quad (2.4)$$

Where:

- $P(a|s) \in [0, 1]$ is the 0-node Policy prior probability assigned to move a by the Transformer network.
- $\sum_b N(s, b) = N(s)$ is the total visit count of the parent node s .
- $N(s, a)$ is the visit count of candidate move a . The denominator $(1 + N(s, a))$ ensures $U(s, a)$ shrinks as move a is visited more frequently.
- c_{puct} is a scaling constant (typically $c_{\text{puct}} \approx 1.25 + \frac{\ln((N(s) + c_{\text{base}} + 1)/c_{\text{base}})}{c_{\text{init}}}$) that slowly increases exploration as total visits grow.

2.3 Asymptotic Analysis & The Refutation Mechanism

The interaction between $Q(s, a)$ and $U(s, a)$ yields critical theoretical properties:

Property 1: Policy Dominance at Low Visit Counts ($N(s, a) \rightarrow 0$)

When a position is first expanded, $N(s, a) = 0$ for all moves, making $U(s, a) \gg Q(s, a)$. The search priority is almost entirely dictated by $P(a|s)$. LC0 allocates 99% of its initial calculations to top-policy moves.

Property 2: Value Convergence at High Visit Counts ($N(s, a) \rightarrow \infty$)

As total search visits grow, $N(s, a)$ increases, driving $U(s, a) \rightarrow 0$. The selection rule converges to pure empirical action value:

$$\lim_{N(s) \rightarrow \infty} a^* = \arg \max_a Q(s, a) \quad (2.5)$$

This guarantees that given sufficient compute, MCTS will find the objectively correct move even if the neural policy initially disliked it.

Property 3: Automatic Refutation & Searching Pivots

Suppose move a_{natural} has a very high policy prior ($P(a_{\text{natural}}|s) = 0.80$). MCTS immediately investigates a_{natural} . However, if deep search reveals a hidden tactical refutation, the backpropagated scores $V(s')$ will be catastrophic, causing $Q(s, a_{\text{natural}})$ to collapse toward -1.0 .

As $Q(s, a_{\text{natural}})$ drops, the total sum $Q(s, a_{\text{natural}}) + U(s, a_{\text{natural}})$ falls below the priority of an alternative move a_{alt} (which has a lower prior $P(a_{\text{alt}}|s) = 0.15$ but $N(s, a_{\text{alt}}) = 0$). The PUCT formula automatically triggers a **search pivot**, abandoning the failed natural move to explore candidate a_{alt} .

2.4 Intuitive Analogies of the PUCT Formula

2.4.1 The Casino Slot Machine Model

In a casino with 50 slot machines:

- $P(a|s)$ is your initial intuition about which machines look lucky before playing.
- $Q(s, a)$ is your actual average payout from playing Machine A.
- $N(s, a)$ is how many times you've pulled Machine A's lever.
- The PUCT formula forces you to start playing the most promising-looking machines (P), but if Machine A stops paying out (Q drops), you pivot to explore unplayed machines ($N = 0$).

2.4.2 The Talent Hiring / Job Interview Model

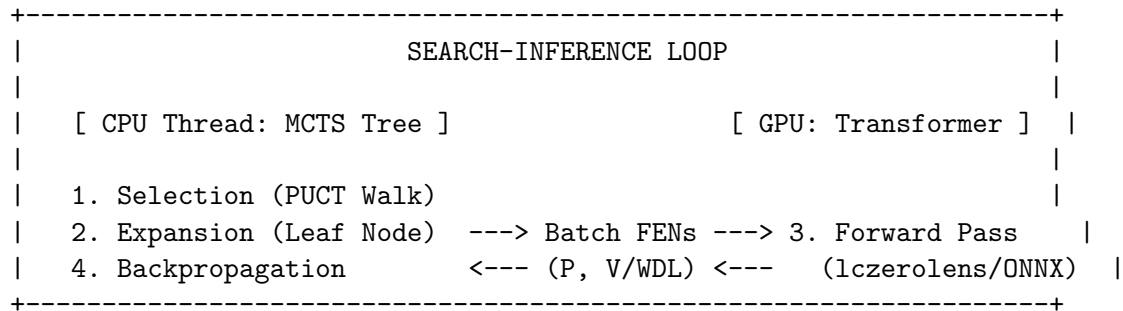
A hiring manager has 100 job applicants for 1 open position and a budget of 50 total interview slots:

$$\text{Priority} = \underbrace{Q(\text{Interview Score})}_{\text{Verified Technical Performance}} + c_{\text{puct}} \cdot \underbrace{P(\text{Resume Score})}_{\text{Policy Prior Screening}} \cdot \frac{\sqrt{\text{Total Interviews Conducted}}}{1 + \text{Rounds Done with Applicant}} \quad (2.6)$$

- **Resume Screening (P):** Applicant A (ex-Google, 10 yrs exp) gets $P_A = 70\%$. Applicant B gets $P_B = 20\%$. Applicant A gets Round 1 first.
- **Interviewing (Q):** Applicant A aces Round 1 ($Q_A = 0.95$). Because Q_A is high, Applicant A is invited to Rounds 2 and 3.
- **Bombing Round 3 (Refutation):** Applicant A fails the live coding test (Q_A collapses to 0.10).
- **The Pivot:** Applicant B has 0 rounds done ($N_B = 0$), giving Applicant B a large Uncertainty Bonus. The hiring manager immediately stops interviewing Applicant A and calls Applicant B.

2.5 The 4-Phase Search-Inference Loop

The physical execution of LC0 involves continuous communication between the CPU (managing the MCTS tree) and the GPU (executing Transformer forward passes):



Phase	Engine Component	Mathematical Computation
1. Selection	CPU (C++ Engine Pool)	Walk down tree maximizing $Q(s, a) + U(s, a)$ until reaching an unexpanded leaf s_L .
2. Expansion	CPU	Create child node s_L by applying move a .
3. Inference	GPU (PyTorch/ONNX via <code>lczerolens</code>)	Run batched forward pass on s_L tensor to compute Policy $P(a' s_L)$ and Value $\mathbf{v}(s_L)$.
4. Backprop	CPU	Propagate $V(s_L)$ back up the path, updating $N(s, a) \leftarrow N(s, a) + 1$ and $W(s, a) \leftarrow W(s, a) + V(s_L)$.

Table 2.1: The 4-Phase Execution Pipeline

Chapter 3

Mechanistic Interpretability & Linear Probing

3.1 Peering Inside the Black Box: Intermediate Transformer Layers

While standard engine wrappers only output the final Policy $P(a|s)$ and Value $V(s)$ from Layer 15, mechanistic interpretability seeks to read the **internal computational state** of the network at intermediate layers $l \in \{1, \dots, 14\}$.

Using PyTorch layer hooks via `lczeroLens` (see `backend/neural_vision.py`), we extract the internal spatial activation tensor $\mathbf{X}^{(l)} \in \mathbb{R}^{64 \times 768}$ during a forward pass.

3.2 Mathematical Formulation of Linear Probes

A **Linear Probe** is a parameter-efficient classifier $\mathbf{W}_{\text{probe}} \in \mathbb{R}^{K \times 768}$ and bias vector $\mathbf{b} \in \mathbb{R}^K$ trained on frozen intermediate layer representations $\mathbf{X}^{(l)}$:

$$\hat{\mathbf{y}} = \sigma \left(\mathbf{W}_{\text{probe}} \mathbf{x}_i^{(l)} + \mathbf{b} \right) \quad (3.1)$$

Where $\mathbf{x}_i^{(l)} \in \mathbb{R}^{768}$ is the activation vector at square token i at layer l , and σ is the sigmoid or softmax activation function.

3.2.1 Finding 1: Implicit Future Look-Ahead (3–7 Plies Ahead)

Research on chess transformer models (*Evidence of Learned Look-Ahead in a Chess-Playing Neural Network*) reveals an extraordinary phenomenon: **middle layers (Layers 6–10) implicitly construct spatial board representations of future positions 3 to 7 plies ahead inside a single forward pass**, without any external MCTS tree search.

By training a linear probe on Layer 8 activations, the probe can predict the optimal piece placement 4 plies into the future with up to $\approx 92\%$ accuracy:

$$P(\text{Piece } p \text{ on square } j \text{ at ply } t + 4) = \sigma \left(\mathbf{W}_{\text{future}}^{(8)} \mathbf{X}^{(8)} \right) \quad (3.2)$$

3.2.2 Finding 2: Time-Traveling Attention Heads

By isolating specific attention heads h at layer l , we discover "look-ahead attention heads" whose query-key pairs $\mathbf{Q}_h^{(l)}(\mathbf{K}_h^{(l)})^T$ route information **forward and backward in time**. A head at Layer 7 attends to squares where pieces *will be* located after a tactical exchange, routing that future structural information back to shape the immediate move selection at Layer 1.

3.3 The Suppressed-Win Probe ("The Tal Detector")

The most profound finding for our training tool is **Prior Override of Look-Ahead** (the "forgotten puzzle" phenomenon).

The Mechanism of Suppressed Sacrifices:

At intermediate layers (Layers 7–9), the transformer’s hidden representations successfully discover a winning sacrificial combination or forced checkmate sequence. The linear probe at Layer 8 outputs a high probability of tactical victory ($\hat{y}_{\text{tactical}}^{(8)} > 0.90$).

However, as the activations flow through deep layers (Layers 10–15), **deep training priors overpower the intermediate discovery**. The network’s final output Policy Head $P(a|s)$ penalizes the sacrifice because giving up material looks statically risky based on millions of self-play games. The network "talks itself out" of the winning sacrifice, outputting a safe, passive move instead.

Our system implements a **Suppressed-Win Probe** by comparing Layer 8 probe predictions against Layer 15 Policy outputs:

$$\Delta_{\text{suppression}} = \hat{y}_{\text{tactical}}^{(8)} - P(a_{\text{sac}}|s) \quad (3.3)$$

When $\Delta_{\text{suppression}} > \tau$, the tool flags a **"Tal Sacrifice Suppressed by LCO"**, surfacing the move for the user as an aspirational training drill!

Chapter 4

Advanced Engine Steering & Reinforcement Learning

4.1 Hacking MCTS Search vs. Representation Steering

A fundamental architectural debate in AI chess design is how to steer an engine toward specific playing styles (e.g., sharp sacrificial play, heavy pin/skewer configurations, or suffocating positional restriction).

4.1.1 Search Hacking: Heuristic PUCT Bias

One approach is modifying the PUCT selection formula at runtime by injecting an auxiliary motif score $M_{\text{motif}}(PV)$ computed over the Principal Variation (PV) of searched nodes:

$$Q_{\text{biased}}(s, a) = Q(s, a) + \lambda \cdot M_{\text{motif}}(PV(s, a)) \quad (4.1)$$

Drawback: Running complex feature taggers over millions of MCTS nodes per second introduces massive latency and risks the **Horizon Effect**, where the engine blunders material just to trigger a meaningless tactical tag.

4.1.2 Multi-Objective Reinforcement Learning

Instead of hacking search, Schrittwieser et al. proposed adding an Auxiliary Motif Output Head to the neural network alongside the Value Head:

$$\mathbf{v}_{\text{multi}}(s) = (V_{\text{win}}(s), V_{\text{motif}}(s)) \quad (4.2)$$

At inference time, the user adjusts a dial $\alpha \in [0, 1]$ to set the effective evaluation:

$$V_{\text{steered}}(s) = (1 - \alpha)V_{\text{win}}(s) + \alpha V_{\text{motif}}(s) \quad (4.3)$$

This allows seamless style adjustment without corrupting the core win-rate evaluation.

4.2 Creating Tactical Chaos: Intrinsic Motivation & Entropy

Mikhail Tal famously remarked that chess is not a math problem, but an attack on the opponent's psychology. To create an engine that plays like Tal, we formalize "discomfort" using **Intrinsic Motivation** driven by **Policy Entropy**.

When LC0 evaluates a candidate position s' , it predicts the opponent's response distribution $P(a_{\text{opp}}|s')$. The **Policy Entropy** $H(P_{s'})$ measures the opponent's uncertainty:

$$H(P_{s'}) = - \sum_{a_{\text{opp}}} P(a_{\text{opp}}|s') \ln P(a_{\text{opp}}|s') \quad (4.4)$$

- **Low Entropy (Quiet Position):** The opponent has 1 obvious, easy response ($P = 95\%$, $H \rightarrow 0$).
- **High Entropy (Nightmare Minefield):** The opponent faces 10 razor-sharp moves where a single mistake loses instantly, causing the policy distribution to flatten ($H \gg 0$).

By modifying the steering score to maximize opponent policy entropy:

$$\text{Score}_{\text{Tal}}(s') = V_{\text{win}}(s') + \gamma \cdot H(P_{s'}) \quad (4.5)$$

The engine relentlessly seeks out high-complexity tactical minefields. Sacrifices and wild attacks emerge spontaneously as the optimal mathematical solution to maximizing opponent chaos!

4.3 Creating Positional Paralyzation: Dense Mobility Rewards

Conversely, to model Magnus Carlsen’s ”anaconda” style, we reward the engine for restricting enemy piece mobility.

Let $\mathcal{M}_{\text{opp}}(s')$ be the number of legal squares available to the opponent in position s' . We define a **Dense Mobility Restriction Reward**:

$$R_{\text{mobility}}(s') = -\lambda \cdot |\mathcal{M}_{\text{opp}}(s')| \quad (4.6)$$

When integrated into MCTS trajectory rollouts, the engine prioritizes lines that trap, pin, and suffocate enemy pieces. Pins and skewers naturally emerge as the geometric consequence of mobility restriction.

4.4 Latent World Models & Guided Trajectory Rollouts

In modern architectures (e.g., MuZero / Dreamer), the engine simulates future trajectories in a **Latent Space** $\mathbf{z}_{t+1} = g(\mathbf{z}_t, a_t)$.

Using **Guided Trajectory Rollouts**, the engine simulates 100,000 future paths in its latent imagination. Instead of selecting the highest raw win-rate path, it applies a **Style Constraint Filter**:

$$\text{Path}^* = \arg \max_{\text{Path} \in \text{Futures}} V(\text{Path}) \quad \text{subject to} \quad \text{StyleScore}(\text{Path}) \geq \theta \quad (4.7)$$

This guarantees that the engine plays objectively winning chess while strictly adhering to the user’s selected tactical or structural training style.

Chapter 5

Human-AI Translation & The Saliency Problem

5.1 The Failure of the Single Evaluation Bar

For 30 years, chess software displayed a single scalar bar (e.g., +1.4) alongside a raw move variation. To a human student (rated ~ 2100), a score of +1.4 provides zero actionable learning value. It states *that* a position is winning, but fails to explain *why*.

Our platform replaces the scalar bar with a multi-channeled **Neural Visualization Layer**:

1. **Spatial Attention Heatmaps:** Rendering layer-averaged attention $\mathbf{A}^{(l)}$ directly over board squares via CSS overlays.
2. **Policy Disparity Vectors:** Displaying candidate move arrows colored by raw policy probability $P(a|s)$.
3. **WDL Atmospheric Lighting:** Coloring the UI border based on position entropy $H(\mathbf{v}(s))$.

5.2 The Policy-MCTS Divergence Index ($\Delta_{\text{Policy, MCTS}}$)

By comparing the 0-node Policy prior $P(a|s)$ against the final MCTS visit distribution $\pi_{\text{MCTS}}(a|s) = \frac{N(s,a)}{\sum_b N(s,b)}$, we define the **Divergence Index**:

$$\Delta_{\text{Policy, MCTS}}(a) = P(a|s) - \pi_{\text{MCTS}}(a|s) \quad (5.1)$$

This classifies positions into three pedagogical categories:

Category	$P(a s)$	$\pi_{\text{MCTS}}(a s)$	Pedagogical Meaning
Solid Play	High (> 0.50)	High (> 0.50)	Natural, sound move. Intuition and calculation agree.
Optical Trap	High (> 0.60)	Low (< 0.05)	Blunder Trap! Move looks tempting to human/neural instinct, but fails to deep calculation.
Hidden Gem	Low (< 0.05)	High (> 0.60)	Grandmaster Move! Counter-intuitive key move requiring deep calculation.

Table 5.1: Classification via Policy-MCTS Divergence

5.3 Relational Board Facts & The Saliency Problem

To generate natural language explanations without LLM chess hallucination, our backend (`backend/training/relational_facts.py`) extracts deterministic, verified board facts over LC0’s principal variations:

- **Tactical Facts:** Absolute/relative pins, x-rays, defender removal, protected passed pawns, knight outposts.
- **Positional Facts:** Isolated/doubled/backward pawns, bad bishops, open files, 7th-rank rook placement.

5.3.1 The Saliency Problem Defined

A major challenge arises: for any position, the deterministic extractor outputs dozens of true board facts (e.g., "pawn on a3 is backward", "rook controls c-file", "knight pinned on f6"). Listing all true facts creates unreadable noise.

The Saliency Problem: *Which two or three extracted facts represent THE actual objective of the position?*

5.3.2 Contrastive Objective Extraction

We solve saliency by computing facts **contrastively** between the best line a^* and inferior alternative lines a_{alt} :

$$\text{Salient Facts}(a^*) = \text{Facts}(a^*) \setminus \text{Facts}(a_{\text{alt}}) \quad (5.2)$$

The objective of a move is uniquely defined by what the correct continuation achieves that inferior alternatives fail to accomplish (e.g., "Move a^* removes the defender of the dark squares, whereas move a_{alt} leaves the dark squares weak").

5.3.3 Learning Saliency from Grandmaster Annotations

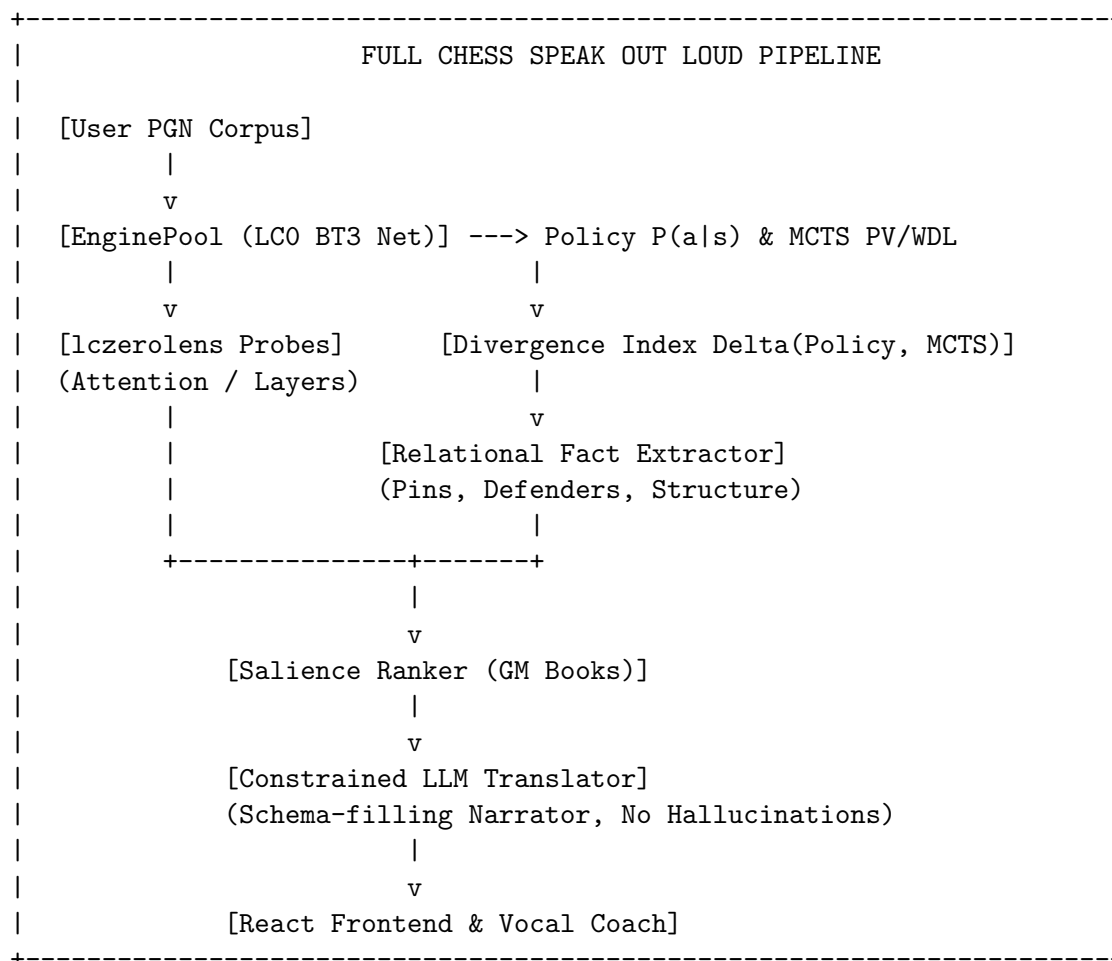
To refine saliency ranking, we pair contrastive facts with a public-domain corpus of grandmaster annotations (Steinitz, Capablanca, Alekhine). A master’s textual comment on a position serves as a ground-truth label of what was salient. A supervised ranking model learns to score facts based on their alignment with master annotations, creating an accurate, non-hallucinating natural language coach!

Chapter 6

System Architecture of the AI Coach

6.1 System Dataflow Pipeline

The overall platform architecture integrates LC0 engine evaluation, PyTorch interpretability probes, deterministic fact extraction, and a constrained LLM translator:



6.2 The Translator’s Contract (Zero Hallucination Guarantee)

To adhere to the foundational project rule—“*LC0 is the coach; the LLM is a translator of LC0’s thoughts, never a chess reasoner*”—the LLM operates under a strict API contract:

1. The LLM receives a structured JSON payload containing **only** verified relational facts, PV lines, and WDL values.
2. The LLM is forbidden from calculating moves or evaluating positions independently.
3. If evidence is ambiguous, the LLM must emit concise, factual descriptions or remain silent.
4. Every verbal claim must cite a specific board square, line, or verified fact present in the input packet.

6.3 Conclusion

By combining deep neural network intuition (Policy), MCTS calculation (PUCT), mechanistic interpretability (Linear Probes), contrastive fact extraction, and learned grandmaster salience, the **Chess Speak Out Loud** architecture transforms LC0 from an alien calculation engine into an intimate, position-grounded AI chess coach.